

AI ASSISTED CODING

ASSIGNMENT- 5.1 & 6

NAME: Varshitha. G

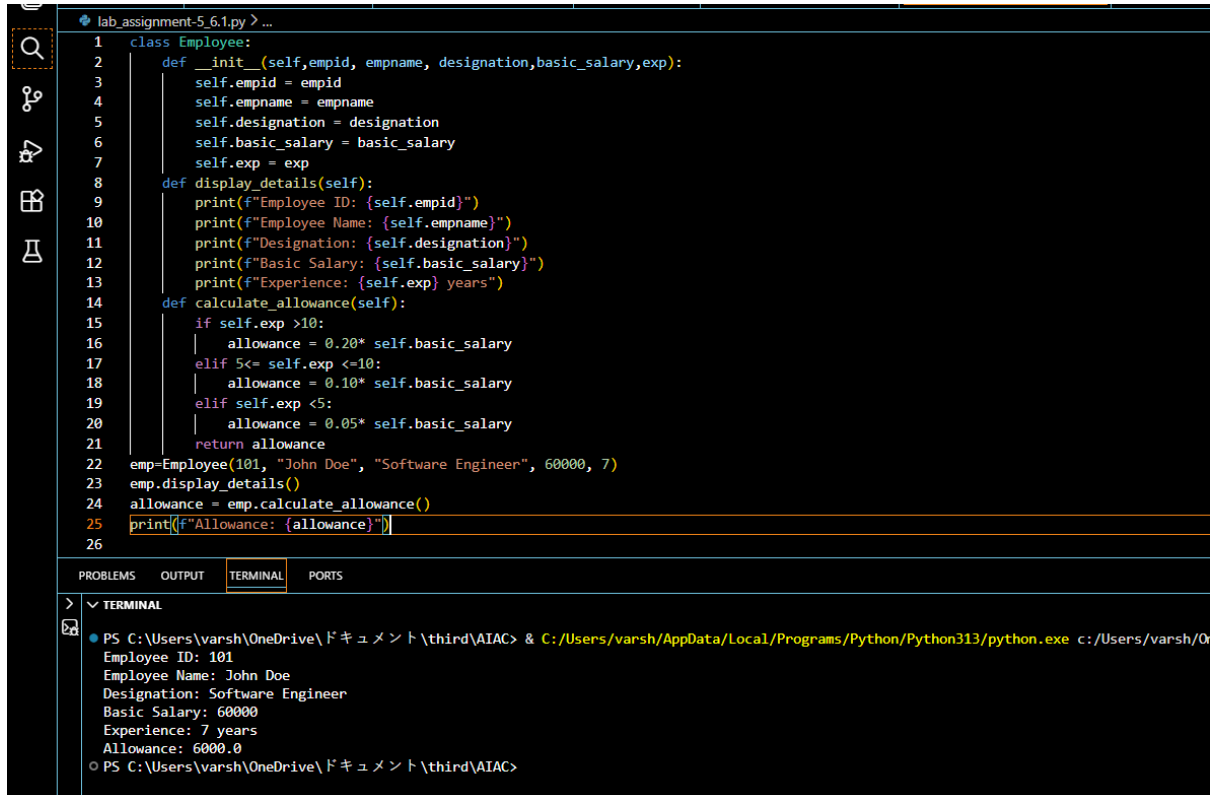
HTNO: 2303A51103

Task 1:

Employee Data: Create Python code that defines a class named 'Employee' with the following attributes: 'empid', 'empname', 'designation', 'basic_salary', and 'exp'. Implement a method 'display_details()' to print all employee details. Implement another method 'calculate_allowance()' to determine additional allowance based on experience:

- If 'exp > 10 years' → allowance = 20% of 'basic_salary'
- If '5 ≤ exp ≤ 10 years' → allowance = 10% of 'basic_salary'
- If 'exp < 5 years' → allowance = 5% of 'basic_salary'

Finally, create at least one instance of the 'Employee' class, call the 'display_details()' method, and print the calculated allowance.



```
lab_assignment-5.6.1.py > ...
1 class Employee:
2     def __init__(self, empid, empname, designation, basic_salary, exp):
3         self.empid = empid
4         self.empname = empname
5         self.designation = designation
6         self.basic_salary = basic_salary
7         self.exp = exp
8     def display_details(self):
9         print(f"Employee ID: {self.empid}")
10        print(f"Employee Name: {self.empname}")
11        print(f"Designation: {self.designation}")
12        print(f"Basic Salary: {self.basic_salary}")
13        print(f"Experience: {self.exp} years")
14    def calculate_allowance(self):
15        if self.exp > 10:
16            allowance = 0.20 * self.basic_salary
17        elif 5 <= self.exp <= 10:
18            allowance = 0.10 * self.basic_salary
19        elif self.exp < 5:
20            allowance = 0.05 * self.basic_salary
21        return allowance
22 emp = Employee(101, "John Doe", "Software Engineer", 60000, 7)
23 emp.display_details()
24 allowance = emp.calculate_allowance()
25 print(f"Allowance: {allowance}")
26
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive\ドキュメント\third\AIAC/lab_assignment-5.6.1.py
Employee ID: 101
Employee Name: John Doe
Designation: Software Engineer
Basic Salary: 60000
Experience: 7 years
Allowance: 6000.0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

- Units $\leq 100 \rightarrow ₹5$ per unit
- 101 to 300 units $\rightarrow ₹7$ per unit
- More than 300 units $\rightarrow ₹10$ per unit

Create a bill object, display details, and print the total bill amount.

```
25 print(f"Allowance: {allowance}")'''
26
27 class ElectricityBill:
28     def __init__(self, customer_id, name, units_consumed):
29         self.customer_id = customer_id
30         self.name = name
31         self.units_consumed = units_consumed
32     def display_details(self):
33         print(f"Customer ID: {self.customer_id}")
34         print(f"Customer Name: {self.name}")
35         print(f"Units Consumed: {self.units_consumed}")
36     def calculate_bill(self):
37         if self.units_consumed <= 100:
38             bill_amount = self.units_consumed * 5
39         elif 101 <= self.units_consumed <= 300:
40             bill_amount = (100 * 5) + (self.units_consumed - 100) * 7
41         else:
42             bill_amount = (100 * 5) + (100 * 7) + (self.units_consumed - 300) * 10
43         return bill_amount
44 bill = ElectricityBill(201, "Alice Smith", 350)
45 bill.display_details()
46 bill_amount = bill.calculate_bill()
47 print(f"Total Bill Amount: {bill_amount}")
48
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

```
● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-5_6.1.py
Customer ID: 201
Customer Name: Alice Smith
Units Consumed: 350
Total Bill Amount: 1700
○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details. Implement another method `calculate\_discount()` where:

- Electronics → 10% discount
- Clothing → 15% discount
- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.

```
49 class Product:
50     def __init__(self, product_id, product_name, price, category):
51         self.product_id = product_id
52         self.product_name = product_name
53         self.price = price
54         self.category = category
55     def display_details(self):
56         print(f"Product ID: {self.product_id}")
57         print(f"Product Name: {self.product_name}")
58         print(f"Price: {self.price}")
59         print(f"Category: {self.category}")
60     def calculate_discount(self):
61         if self.category.lower() == "electronics":
62             discount = 0.10 * self.price
63         elif self.category.lower() == "clothing":
64             discount = 0.15 * self.price
65         elif self.category.lower() == "groceries":
66             discount = 0.05 * self.price
67         return discount
68 product = Product(301, "Smartphone", 50000, "Electronics")
69 product.display_details()
70 discount = product.calculate_discount()
71 print(f"Final Price: {product.price - discount}")
72 product= Product(202, "Jeans", 2000, "Clothing")
73 product.display_details()
74 discount = product.calculate_discount()
75 print(f"Final Price: {product.price - discount}")
```

PROBLEMS OUTPUT **TERMINAL** PORTS ...

> **TERMINAL**

```
Category: Clothing
Final Price: 1700.0
Product ID: 301
Product Name: Smartphone
Price: 50000
Category: Electronics
Final Price: 45000.0
Product ID: 202
Product Name: Jeans
Price: 2000
Category: Clothing
Final Price: 1700.0
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee.

```
78 class LibraryBook:
79     def __init__(self,book_id,title,author,borrower,days_late):
80         self.book_id = book_id
81         self.title = title
82         self.author = author
83         self.borrower = borrower
84         self.days_late = days_late
85     def display_details(self):
86         print(f"Book ID: {self.book_id}")
87         print(f"Title: {self.title}")
88         print(f"Author: {self.author}")
89         print(f"Borrower: {self.borrower}")
90         print(f"Days Late: {self.days_late}")
91     def calculate_late_fee(self):
92         if self.days_late <=5:
93             late_fee = self.days_late * 5
94         elif 6 <= self.days_late <=10:
95             late_fee = self.days_late * 7
96         else:
97             late_fee = self.days_late * 10
98         return late_fee
99 book = LibraryBook(401, "The Great Gatsby", "F. Scott Fitzgerald", "Bob Johnson", 8)
100 book.display_details()
101 late_fee = book.calculate_late_fee()
102 print(f"Late Fee: {late_fee}")
```

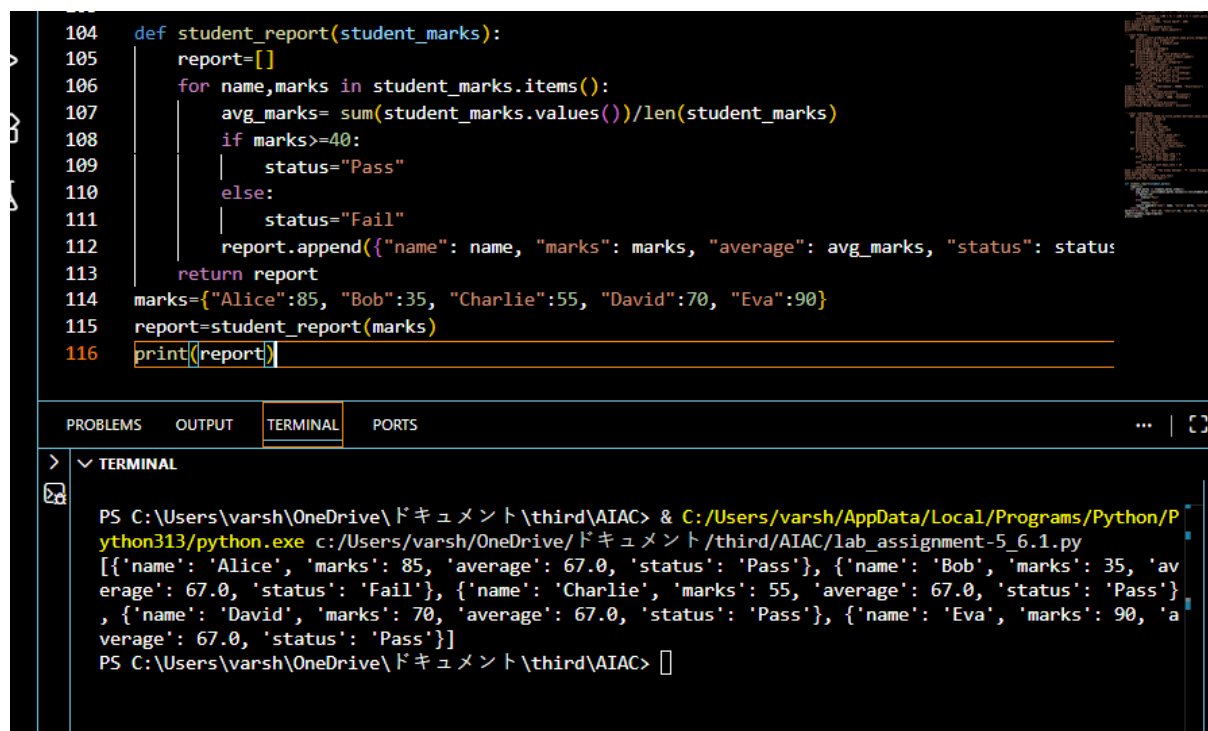
PROBLEMS	OUTPUT	TERMINAL	PORTS
> TERMINAL Category: Clothing Product ID: 202 Product Name: Jeans ... ● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-5_6.1.py Book ID: 401 Title: The Great Gatsby Author: F. Scott Fitzgerald Borrower: Bob Johnson Days Late: 8 Late Fee: 56 ○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █			

Task 5:

Student Performance Report - Define a function `student\_report(student\_data)` that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.



```
104 def student_report(student_marks):
105     report=[]
106     for name,marks in student_marks.items():
107         avg_marks= sum(student_marks.values())/len(student_marks)
108         if marks>=40:
109             status="Pass"
110         else:
111             status="Fail"
112         report.append({"name": name, "marks": marks, "average": avg_marks, "status": status})
113     return report
114 marks={"Alice":85, "Bob":35, "Charlie":55, "David":70, "Eva":90}
115 report=student_report(marks)
116 print(report)
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-5_6.1.py
[{'name': 'Alice', 'marks': 85, 'average': 67.0, 'status': 'Pass'}, {'name': 'Bob', 'marks': 35, 'average': 67.0, 'status': 'Fail'}, {'name': 'Charlie', 'marks': 55, 'average': 67.0, 'status': 'Pass'}, {'name': 'David', 'marks': 70, 'average': 67.0, 'status': 'Pass'}, {'name': 'Eva', 'marks': 90, 'average': 67.0, 'status': 'Pass'}]
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

```
117
118 class TaxiRide:
119     def __init__(self,ride_id,driver_name,distance_km,waiting_time_min):
120         self.ride_id = ride_id
121         self.driver_name = driver_name
122         self.distance_km = distance_km
123         self.waiting_time_min = waiting_time_min
124     def display_details(self):
125         print(f"Ride ID: {self.ride_id}")
126         print(f"Driver Name: {self.driver_name}")
127         print(f"Distance (km): {self.distance_km}")
128         print(f"Waiting Time (min): {self.waiting_time_min}")
129     def calculate_fare(self):
130         if self.distance_km <=10:
131             fare = self.distance_km * 15
132         elif 11 <= self.distance_km <=30:
133             fare = (10 * 15) + (self.distance_km - 10) * 12
134         else:
135             fare = (10 * 15) + (20 * 12) + (self.distance_km - 30) * 10
136         fare += self.waiting_time_min * 2
137         return fare
138 ride = TaxiRide(501, "Charlie Brown", 25, 10)
139 ride.display_details()
140 fare = ride.calculate_fare()
141 print(f"Total Fare: {fare}")
```

PROBLEMS OUTPUT **TERMINAL** PORTS ..

> **TERMINAL**

```
Ride ID: 501
Driver Name: Charlie Brown
Distance (km): 25
Waiting Time (min): 10
Total Fare: 350
PS C:\Users\varsh\OneDrive\Documents> python3\ATACs
```

Task 7:

Statistics Subject Performance - Create a Python function

`statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

```
142
143 def statistics_subject(score_list):
144     total = sum(score_list)
145     average = total / len(score_list)
146     highest = max(score_list)
147     lowest = min(score_list)
148     passed = 0
149     failed=0
150     for i in score_list:
151         if i>=40:
152             passed+=1
153         else:
154             failed+=1
155     print(f"Number of Students Passed: {passed}")
156     print(f"Number of Students Failed: {failed}")
157     return { "average": average, "highest": highest, "lowest": lowest}
158 scores = [ 28, 49, 33, 72, 15, 60, 95, 40, 53, 81, 22, 47, 68, 79, 34, 91, 44, 58, 73]
159 stats = statistics_subject(scores)
160 print(stats)
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> ▼ TERMINAL

b_assignment-5_6.1.py
Number of Students Passed: 46
Number of Students Failed: 14
{'average': 58.6, 'highest': 95, 'lowest': 15}
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

.

```
162 # Generate two programs naive approach and optimized approach to check if given number is prime or not also calculate time and space complexities of both programs
163 import time
164 # Naive Approach
165 def is_prime_naive(n):
166     if n <= 1:
167         return False
168     for i in range(2, n):
169         if n % i == 0:
170             return False
171     return True
172 start_time = time.time()
173 number = 29
174 result_naive = is_prime_naive(number)
175 end_time = time.time()
176 print(f"Naive Approach: Is {number} prime? {result_naive}")
177 print(f"Time taken (Naive): {end_time - start_time} seconds")
178 # Optimized Approach
179 def is_prime_optimized(n):
180     if n <= 1:
181         return False
182     if n <= 3:
183         return True
184     if n % 2 == 0 or n % 3 == 0:
185         return False
186     i = 5
187     while i * i <= n:
188         if n % i == 0 or n % (i + 2) == 0:
189             return False
190         i += 6
191     return True
192 start_time = time.time()
193 result_optimized = is_prime_optimized(number)
194 end_time = time.time()
195 print(f"Optimized Approach: Is {number} prime? {result_optimized}")
196 print(f"Time taken (Optimized): {end_time - start_time} seconds")
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
Optimized Approach: Is 29 prime? True
Time taken (Naive): 6.9141387939453125e-06 seconds
Optimized Approach: Is 29 prime? True
Time taken (Optimized): 2.86102294921875e-06 seconds
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

The optimized method performs far fewer checks, especially for large n.

This reduction from linear time to square-root time makes the optimized approach much faster and more efficient in practice.

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

```
217 #generate python code that develop fibonacci sequence using recursion Provide well commented explanation of each step
218 def fibonacci(n):
219     # Base case: if n is 0 or 1, return n
220     if n <= 1:
221         return n
222     else:
223         # Recursive case: return the sum of the two preceding numbers
224         return fibonacci(n - 1) + fibonacci(n - 2)
225 # Number of terms in the Fibonacci sequence
226 num_terms = 10
227 print(f"Fibonacci sequence up to {num_terms} terms:")
228 for i in range(num_terms):
229     print(fibonacci(i), end=" ")
230 # The above code defines a recursive function to generate Fibonacci numbers.
231
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/
Fibonacci sequence up to 10 terms:
0 1 1 2 3 5 8 13 21 34
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```


Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt: “Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

```
229
230 #generate a program that reads a file and process the data Generate code with proper error handling and clear explanatio
231
232 def read_file(file_path):
233     try:
234         # Attempt to open the file
235         with open(file_path, 'r') as file:
236             data = file.read()
237             print("File content successfully read.")
238             return data
239     except FileNotFoundError:
240         # Handle the case where the file does not exist
241         print(f"Error: The file at {file_path} was not found.")
242     except PermissionError:
243         # Handle the case where there are permission issues
244         print(f"Error: You do not have permission to read the file at {file_path}.")
245     except Exception as e:
246         # Handle any other exceptions that may occur
247         print(f"An unexpected error occurred: {e}")
248 file_path = 'example.txt' # Specify the path to your file here
249 file_content = read_file(file_path)
250 if file_content:
251     print("File Content:")
252     print(file_content)
```

PROBLEMS OUTPUT TERMINAL PORTS

> ▼ TERMINAL

```
File content successfully read.
File Content:
File Content:
Hello.. Good Evening..
This is AI assisted coding lab
Lab Assignment 5
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> 
```